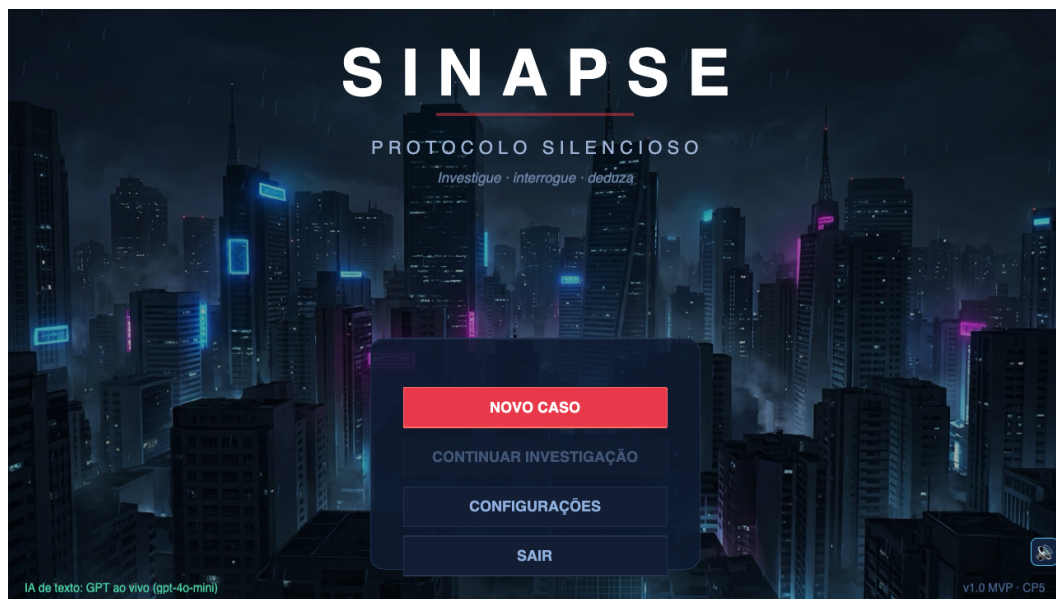


CHECKPOINT 5: VIBE CODING E MVP

SINAPSE

PROTOCOLO SILENCIOSO

MVP jogável de um jogo de investigação narrativa com suspeitos movidos por IA generativa (GPT) e arte gerada no Nano Banana



Menu principal do MVP

Integrantes

Gabriel Palmeira (RM 563522)

Mayara Mota (RM 563887)

FIAP, turma 2TIAPY

NLP, Chatbots e Agentes Virtuais

São Paulo, outubro de 2026

Jogo online: <https://jogo-fiap.onrender.com>

Código: <https://github.com/gabrielpalmeiraa/jogo-fiap>

1. README resumido

O README completo está no Apêndice A. Executar: `python -m venv .env && source .env/bin/activate, pip install -r requirements.txt, cp .env.example .env (colocar OPENAI_API_KEY), python run.py, abrir http://127.0.0.1:8000. Versão pública (build web, jogável sem instalar nada): https://jogo-fiap.onrender.com (a primeira abertura pode levar cerca de 1 minuto, porque o plano gratuito "dorme"). Código: https://github.com/gabrielpalmeiraa/jogo-fiap. Vídeo: [PREENCHER link].`

2. Continuidade com a CP4

Mesmo título, gênero, premissa, elenco, dossiê de evidências e finais da CP4. Mecânicas implementadas (CP4, seção 1.5): interrogatório dinâmico por GPT, medidores de Confiança e Pressão injetados no prompt, coleta e cruzamento de evidências, contador de 7 dias e dedução final. Telas: menu, interrogatório (HUD) e Quadro de Investigação. Extras além do mínimo: 4 mecânicas da CP4, testes automatizados, fallback offline, filtro de saída, efeitos sonoros, limites de custo, build público.

3. Diário de Mudanças CP4 x CP5

Item alterado	O que estava na CP4	O que foi implementado	Justificativa técnica
Persistência do estado	PostgreSQL ou SQLite (seção 4.2)	Dicionário em memória no FastAPI (SESSIONS), com limite de sessões e TTL	Partida curta e sem contas de usuário; um banco só traria migração e configuração sem benefício no MVP. Impacto: reiniciar o servidor apaga as partidas em andamento.
Medidores iniciais	Todos os suspeitos começando em 65/35 (seção 9.2)	Beatriz 65/35, Rafael 30/40, Aurora 50/0	A CP4 usou 65/35 só como exemplo da Beatriz. Rafael precisa de confiança baixa e pressão média para a confissão fazer sentido; Aurora não depende de pressão.
Tons de pergunta	Pergunta livre ou menu de tópicos, sem tons	Perguntar, Pressionar e Acolher alteram confiança e pressão por valores fixos no servidor	A CP4 dizia que cada pergunta altera os medidores, mas sem regra. Tons discretos tornam isso determinístico e testável.
Ações por dia	Não definido	4 ações por dia, pressão cai 10 por dia	Sem um custo por turno, os 7 dias não teriam peso e a pressão cruzaria o limiar sem escolha.
Dia 7	Dia de montar a acusação	Final extra "Prazo Esgotado" se o Dia 7 acabar sem acusação	Todo jogo precisa de condição de derrota por tempo; a CP4 só tinha três finais.
Histórico no prompt	Resumir o histórico a cada turno (seção 2.1)	Envia só as últimas 8 mensagens do suspeito	Resumo exigiria uma 2ª chamada ao modelo por turno (mais custo e latência). A janela fixa resolve o limite de contexto no MVP.
Cache de perguntas repetidas	Cache planejado (seção 2.1)	Não implementado	Como o estado muda a cada turno, quase nenhuma pergunta se repete com o mesmo contexto, então o cache quase não acertaria.
Moderação de saída	Filtro de termos proibidos	Filtro por regex (fuga de personagem, spoiler antes do limiar) com fallback para fala pré-escrita	Cobre os dois riscos descritos na CP4 sem chamar uma API de moderação extra.
Evidência "mandado"	"Autorização/mandado formal"	Nome exibido: "Autorização formal (mandado)"	Apenas padronização do nome. Comportamento igual ao da CP4.
Tecnologia do cliente	Phaser com HUD	Phaser nas cenas e HUD em DOM/HTML sobre o canvas	Texto longo, campo de entrada e rolagem são muito mais simples em DOM do que em Phaser.
Arquitetura do backend	FastAPI com chamadas ao GPT	Igual, com rota separada para cada ação (/api/collect, /api/interrogate etc.)	Regras ficam em game.py (sem rede) para serem testadas sem chamar a API.
Limites de custo	Custo por token citado como risco	Máx. 300 caracteres por pergunta, 40 chamadas ao GPT por sessão, 120 por IP por dia e 1500 no total por dia (depois, falas pré-escritas)	Necessário para publicar o jogo com chave real sem risco de gasto descontrolado.

Item alterado	O que estava na CP4	O que foi implementado	Justificativa técnica
Prompts e forma de uso do Nano Banana	P-IMG-01 a 03	Acrescentados P-IMG-04 (Rafael) e P-IMG-05 (Aurora)	A CP4 listava 3 retratos/cenários para 3 suspeitos e 2 cenários; faltavam dois retratos. As 5 imagens foram geradas no Nano Banana (interface do Gemini, camada gratuita, pois a cota da Gemini API para imagem não estava disponível na conta) e integradas ao jogo.

4. Checklist de testes manuais

Executado em 08/10/2026 contra o servidor real, com o GPT (gpt-4o-mini) ligado. Os itens 2 a 18 foram rodados chamando a API do jogo passo a passo (o jogador faz as mesmas chamadas pela tela). Os itens 1 e 19 dependem da interface e estão descritos abaixo. Os testes automáticos (python -m pytest -q, 38 testes) e o tests/e2e_play.py (partida completa no Chromium até o final Caso Encerrado) também passaram.

#	Mecânica	Esperado	Obtido
1	Menu: Novo Caso	Abre a Cena do Crime no Dia 1	Passou. Novo caso começa no Dia 1 com 4 ações e medidores Beatriz 65/35, Rafael 30/40, Aurora 50/0. O menu e a tela de jogo foram vistos nos prints do teste no navegador (docs/prints).
2	Coletar log e e-mail	Evidências aparecem na aba, sem gastar ação	Passou. As duas evidências foram coletadas e as ações continuaram em 4.
3	Mandado antes do Dia 4	Bloqueado	Passou. Resposta 400: "Essa evidência ainda não está disponível." No Dia 4 a coleta funcionou.
4	Perguntar a Beatriz	Resposta do GPT com rótulo "gerado em tempo real"	Passou. A resposta veio com fonte gpt e modelo gpt-4o-mini, em personagem, formal e defensiva.
5	Pressionar	Confiança cai 8, pressão sobe 12	Passou. (68, 38) para (60, 50).
6	Acolher	Confiança sobe 10, pressão cai 4	Passou. (60, 50) para (70, 46).
7	Apresentar evidência	Pressão sobe uma única vez por prova	Passou. Pressão 35 para 53 (tom +3 e log +15). Na segunda apresentação do mesmo log subiu só 3 (56).
8	Beatriz com pressão >= 80	Revela Rafael	Passou. Com pressão 82 ela liberou o segredo e o GPT respondeu "O nome que você procura é Rafael Advir".
9	Rafael com pressão >= 70 ou citando Beatriz com prova	Confessa e ganha a evidência	Passou. Citando Beatriz com o e-mail, Rafael confessou ("Fui eu quem copiou os dados...") e a Confissão entrou nas evidências.
10	Aurora sem mandado	Recusa	Passou. Não liberou e pediu a autorização formal.
11	Aurora com mandado (Dia 4+)	Libera logs	Passou. Liberou, informou que a cópia partiu da conta de Rafael Advir, e os Logs da Aurora entraram nas evidências.
12	4 ações por dia	Na 5ª, pede para encerrar o dia	Passou. A 5ª pergunta voltou 400: "Sem ações hoje. Encerre o dia para continuar."
13	Encerrar dia	Dia +1, pressão -10	Passou. Três vezes levou ao Dia 4, a pressão do Rafael caiu de 70 para 33 em três dias, e o evento do mandado apareceu.
14	Acusar Rafael com 2 provas fortes	Final Caso Encerrado	Passou. Com e-mail, confissão e logs: caso_encerrado.
15	Acusar Rafael com menos provas	Acusação Frágil	Passou. Com uma prova forte: acusacao_fragil.
16	Acusar Beatriz ou Aurora	Investigação Encerrada com Erro	Passou. As duas acusações deram investigacao_erro.
17	Dia 7 encerrado sem acusar	Prazo Esgotado (pede confirmação)	Passou. Encerrar sete dias seguidos deu prazo_esgotado. A confirmação na tela do Dia 7 está no código, mas não foi clicada nesta rodada.
18	Sem chave ou com a API fora do ar	Fala pré-escrita e aviso na tela	Passou. Sem chave: fonte fallback, nota "sem OPENAI_API_KEY". Com chave inválida: fonte fallback, nota "erro da API: AuthenticationError".
19	Botão de som	Liga e desliga os efeitos	Pendente. O som não pôde ser ouvido nesta rodada. Teste no navegador e preencha.

Itens a conferir pelo grupo antes de entregar: o 19 (som), a confirmação do Dia 7 na tela e o retrato da Aurora na tela do interrogatório.

5. Diário de Vibe Coding

Ferramenta: Claude Code (terminal), conduzido por Gabriel Palmeira. Os prompts abaixo foram enviados de fato e estão copiados do histórico (chaves de API aparecem como [chave omitida]).

5.1 Estratégia: conduzir a IA pela rubrica

Em vez de pedir "faça um jogo", tratamos a IA como uma equipe de desenvolvimento que precisa de briefing. A estratégia teve quatro decisões:

- Briefing antes do código.** A sessão de trabalho começa com um documento de contexto que o grupo escreveu: premissa e regras da CP4 que não podem mudar sem registro, resumo da rubrica e dos descontos, decisões já tomadas, pendências e cuidados técnicos aprendidos. Isso evita que a IA reinvente o jogo e protege a nota de continuidade (2,0 pontos).
- Fonte oficial primeiro.** Antes de pedir qualquer extra, enviamos o relatório da CP4 e o enunciado da CP5. O objetivo era descobrir o que o jogo já cumpre e o que diverge da CP4, para registrar tudo no Diário de Mudanças, já que mudança sem registro desconta de 1 a 2 pontos.
- Rubrica como lista de tarefas.** Pedimos que a IA atendesse todos os critérios, e cada item virou uma entrega verificável (tabela 5.2). Os cinco extras da rubrica foram tratados como escopo, não como bônus.
- Verificar em vez de confiar.** Nada foi marcado como pronto sem teste: 38 testes automatizados, partida completa no navegador, o checklist rodado contra o servidor real com o GPT ligado e quatro rodadas de revisão de segurança, cujos achados foram corrigidos.

5.2 Critério da rubrica, o que fizemos e a evidência

Critério	O que fizemos	Evidência
Continuidade com a CP4 (2,0)	Mesmo título, premissa, elenco, dossiê e finais. 13 divergências registradas com justificativa.	Seção 3 deste documento
MVP jogável (2,5)	Menu, interrogatório com HUD e Quadro de Investigação. Seis mecânicas: interrogatório dinâmico, medidores de confiança e pressão, tons de pergunta, coleta e cruzamento de evidências, 7 dias com 4 ações por dia, acusação final. Quatro finais.	Checklist (seção 4) e <code>tests/e2e_play.py</code>
IA generativa real (2,0)	Texto: GPT em tempo real nas falas dos suspeitos. Imagem: 5 imagens do Nano Banana dentro do jogo.	Rótulo "Texto gerado em tempo real pelo GPT" na tela, <code>frontend/assets/</code>
Diário de Vibe Coding (1,5)	Esta seção, com prompts reais, decisões e explicação do código.	Seção 5
Qualidade geral (1,0)	README, checklist com resultados, código separado em regras, IA e API, vídeo.	<code>README.md</code> , seção 4
Extra: build público	Jogo publicado no Render.	https://jogo-fiap.onrender.com
Extra: 2 modalidades de IA	GPT (texto) e Nano Banana (imagem).	Jogo em execução
Extra: mais mecânicas	6 mecânicas contra o mínimo de 3.	Checklist itens 5 a 13
Extra: testes automatizados	38 testes de regras e API, mais partida completa no navegador.	<code>python -m pytest -q</code>
Extra: polimento	Fallback quando a API falha, filtro de saída, indicador "digitando", transições, efeitos sonoros, limites de custo.	Checklist itens 18 e 19

5.3 Prompts-chave

#	Prompt enviado	O que a IA gerou	Decisão e ajuste do grupo
1	Briefing colado no início: "# Contexto do projeto: Sinapse: Protocolo Silencioso (FIAP, CP5)", com as seções "O jogo (CP4, não pode mudar sem registro no Diário de Mudanças)", "Enunciado da CP5 (resumo)", "Decisões do grupo", "O que já está feito", "Pendências" e "Cuidados técnicos aprendidos".	Assumiu o estado do projeto e devolveu quatro frentes de trabalho ordenadas por risco para a nota (teste com a chave real, imagens, diário, revisão final).	O briefing foi decisão do grupo: fixar o que não pode mudar e registrar tudo o que mudar. Entre as lições técnicas passadas estava a troca de cena do Phaser a partir do DOM, para a IA não repetir esse erro.
2	"Vou te mandar o escopo do projeto todo também" (relatório da CP4 e enunciado da CP5).	Leu os dois documentos e comparou com o jogo. Listou 7 divergências (banco de dados, medidores iniciais, tons, ações por dia, final Prazo Esgotado, janela de contexto, cache) e apontou que gerar as imagens valia o ponto extra de 2 modalidades.	Aceitamos a lista e registramos todas no Diário de Mudanças. Decidimos gerar as imagens para cumprir a segunda modalidade. O RM da Mayara foi corrigido para 563887.
3	"pode tocar o restante do projeto. Só quero que atenta TODOS os critérios para que eu tire 10"	Transformou os cinco extras em tarefas: limites de custo para o build público, Dockerfile e render.yaml, efeitos sonoros e botão de som, 3 testes novos, Diário de Mudanças e checklist.	Conferimos cada linha do Diário de Mudanças contra o código (por exemplo, o cache de perguntas realmente não foi implementado). O botão de som cobria o texto "v1.0 MVP" no menu e foi movido.
4	"Vc n pode acessar o nano banana e gerar essas imagens? Se precisar rodar algo no terminal, me avisa"	Script tools/gerar_imagens.py que chama a Gemini API e usa o retrato da Beatriz como referência de estilo.	A primeira execução falhou por chave inválida (o texto de exemplo foi colado no lugar da chave). Depois a API respondeu 429 de cota, porque a conta não tem cota gratuita de imagem pela API. Mudamos de caminho: geramos as imagens pela interface do Gemini, que era o fluxo da CP4, e registramos isso no Diário de Mudanças.
5	"me mande os prompts que vc gostaria que gerasse a imagem para vc colocar no projeto"	Cinco prompts de imagem: os três da CP4, mantidos sem alteração, e dois novos (Rafael e Aurora) no mesmo estilo, com nome de arquivo e proporção.	Mantivemos os prompts da CP4 por continuidade. Colamos os cinco de uma vez e só uma imagem saiu, então passamos a mandar um por mensagem, sem título, anexando o retrato de referência. O crachá do Rafael saiu com "ENGINEZR"; mantivemos, pois o texto é minúsculo e é recortado no painel.
6	"Coloque a chave da openAI no .env: [chave omitida]"	Criou o .env (ignorado pelo git), subiu o servidor e fez duas chamadas reais ao GPT.	Confirmamos fonte gpt e que a Beatriz não revela o nome do Rafael com pressão baixa, como a CP4 exige. Como a chave ficou exposta na conversa, ela será revogada depois do vídeo e o Render usa uma chave separada, com limite de gasto.
7	"me direcione para eu criar o serviço" (com capturas do Render)	Guia passo a passo: tipo de serviço, Docker, plano gratuito, variáveis de ambiente e health check.	O repositório era privado e a aba de repositório público não o achou, então conectamos a conta do GitHub. O deploy subiu em 44 segundos e a IA conferiu o link pela internet (página, health check, imagens e uma pergunta real ao GPT).
8	"é só para o professor mexer"	Reavaliou o risco de abuso e recomendou não pôr senha no jogo.	Decisão do grupo: deixar o link aberto, porque a rubrica pede build "pronto para ser jogado sem setup local" e uma senha poderia custar o ponto extra. Mantivemos os limites de custo e um limite de gasto na conta da OpenAI.
9	"pode preencher a coluna de obtido"	Rodou cada item do checklist contra o servidor real, com o GPT ligado, e registrou o que aconteceu.	O primeiro script falhou por erro nosso (tentou uma 5ª pergunta no mesmo dia, e o limite de 4 ações bloqueou como deveria). Corrigimos o script. Resultado: 18 de 19 itens passaram. O som ficou para teste manual no navegador.

5.4 Problemas encontrados e como resolvemos

O build público levantou riscos que um jogo local não tem. Os commits passaram por quatro rodadas de revisão automática de segurança, e o que ela apontou foi corrigido e coberto por teste.

Problema encontrado	Risco	Correção
O limite de 40 chamadas ao GPT valia só por sessão	Quem abrisse várias partidas gastava a chave sem teto	Tetos por IP, global por dia e limite de partidas novas por hora
A checagem do limite e a contagem aconteciam em momentos diferentes	Requisições simultâneas passavam do limite	A vaga é reservada com lock antes da chamada, e devolvida só se a API não foi chamada
O IP vinha do primeiro valor de x-forwarded-for	O cliente podia forjar o IP e fugir do limite	Usa o último valor (do proxy do Render), só com TRUST_PROXY=1
Tabela de IPs crescendo sem limite, e depois apagando IPs antigos	Esgotar memória, ou zerar o limite de outro IP	Limite de 5000 IPs com um balde compartilhado de estouro, sem bloqueio total
Sessão descartada no meio da requisição	Erro 500 no servidor	A reserva trata sessão inexistente sem quebrar

Nota: a construção inicial do jogo (regras, backend, interface, testes) foi feita em conversas anteriores, resumidas no briefing do prompt 1. Se o grupo quiser, pode acrescentar aqui os prompts dessa fase, copiando do histórico.

Como o código funciona (texto do grupo)

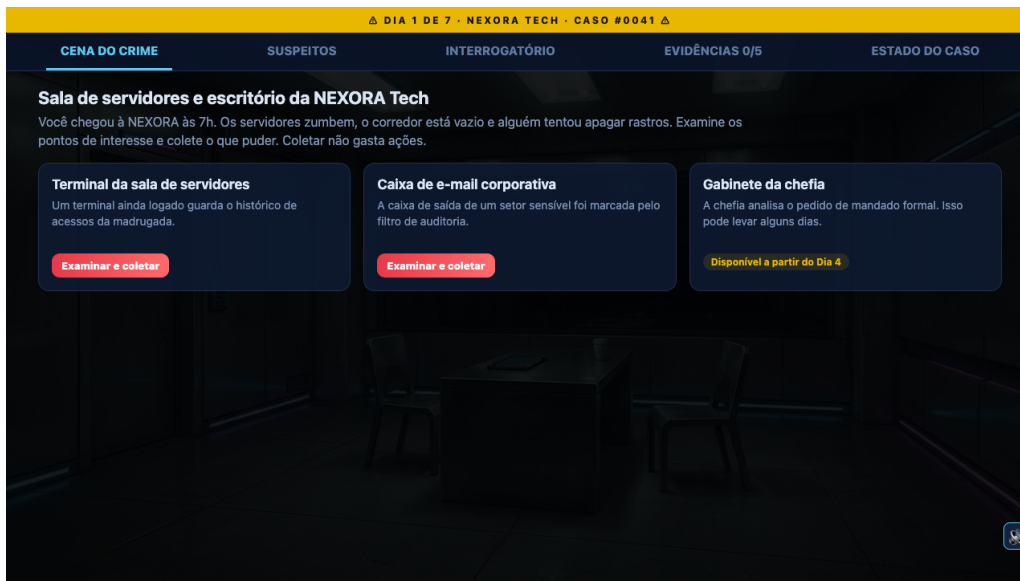
O jogo tem três partes: o servidor com as regras, a integração com o GPT e a tela no navegador. As regras ficam todas em `backend/game.py`, em Python puro, sem rede. É lá que se calculam a confiança e a pressão de cada suspeito, os desbloqueios, as evidências, os dias e os finais. Decidimos assim para que o GPT só escreva a fala do personagem. Se deixássemos o modelo decidir, ele poderia "inventar" que o segredo foi revelado cedo demais. O estado é sempre calculado pelo servidor.

Cada turno de interrogatório segue o mesmo caminho. A tela chama `/api/interrogate` com o suspeito, o tom (perguntar, pressionar ou acolher), as evidências apresentadas e a pergunta. O servidor aplica as regras primeiro: muda confiança e pressão pelo tom, soma a pressão de cada prova apresentada (só uma vez por suspeito) e confere o gatilho do suspeito (Beatriz com pressão de 80% ou mais, Rafael com 70% ou mais ou citando a Beatriz junto de uma prova, Aurora só com o mandado). Só depois disso `backend/llm.py` monta o prompt de sistema com a persona, o segredo, as regras de "bloqueado" ou "liberado" e o estado do caso, que inclui o dia, a confiança, a pressão e as evidências. É assim que os medidores influenciam o GPT, porque entram no texto do prompt a cada turno. O histórico enviado é só o das últimas 8 mensagens para controlar o custo.

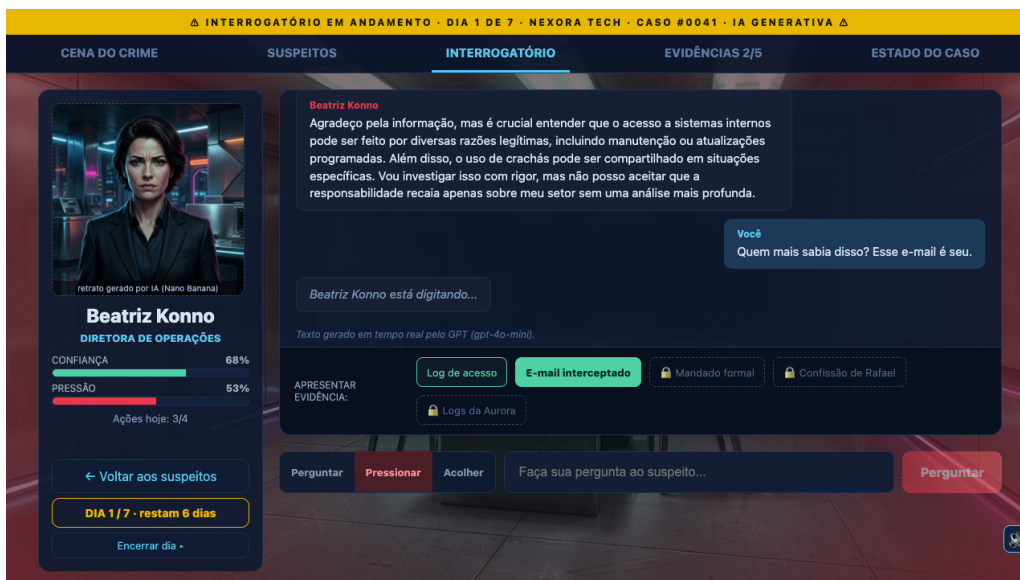
Depois da resposta, passa por um filtro de saída que barra fuga de personagem e spoiler antes do limiar (por exemplo, Rafael confessando antes da hora). Se o filtro bloquear, se não houver chave ou se a API falhar, o jogo usa uma fala pré-escrita (fallback) e avisa na tela. O servidor também limita o gasto: 300 caracteres por pergunta e 40 chamadas ao GPT por partida.

A tela usa Phaser nas cenas (menu, atmosfera do interrogatório e finais) e HTML por cima do canvas para o HUD e o diálogo, porque texto longo e campo de digitação são mais simples em HTML. As imagens geradas no Nano Banana ficam em `frontend/assets/` e o jogo detecta os arquivos sozinho, usando arte procedural se alguma faltar. Os testes automáticos (`python -m pytest -q`) cobrem as regras e a API, e `tests/e2e_play.py` joga uma partida inteira no navegador.

6. Capturas de tela do MVP



Cena do Crime: coleta de evidências (sem gastar ação)



Interrogatório: resposta da Beatriz gerada pelo GPT, com o rótulo de texto em tempo real, retrato do Nano Banana e medidores

DIA 1 DE 7 · NEXORA TECH · CASO #0041

CENA DO CRIME
SUSPEITOS
INTERROGATÓRIO
EVIDÊNCIAS 3/5
ESTADO DO CASO

QUADRO DE INVESTIGAÇÃO: FORMULAR ACUSAÇÃO

Beatriz Konno
Diretora de Operações
PRESSÃO 85% · CONF. 60%

Rafael Advir
Engenheiro Pleno
PRESSÃO 72% · CONF. 22%

Aurora
Assistente de IA corporativa
RESTRITA

FORMULAR ACUSAÇÃO FINAL

Selecione o suspeito e anexe as evidências que sustentam o caso.

ACUSAR:

Beatriz Konno
Rafael Advir
Aurora

EVIDÊNCIAS ANEXADAS:

Log de acesso
E-mail interceptado
Confissão de Rafael
+

ENCERRAR CASO

Esta ação é irreversível.
O desfecho depende de quem você acusar e das evidências que anexar.

Voltar à investigação

Rafael confessou! Nova evidência: Confissão de Rafael.

Quadro de Investigação (acusação final)

FINAL

CASO ENCERRADO

Você acusou Rafael Advir com provas que se sustentam. A NEXORA abriu o processo interno, Rafael perdeu o emprego e Beatriz foi afastada por tentar proteger o subordinado. O vazamento foi contido a tempo.

VOLTAR AO MENU

Final: Caso Encerrado

Sinapse: Protocolo Silencioso | CP5 | FIAP 2TIAPY

Página 8

Apêndice A: README do projeto

Jogo de investigação narrativa em que os suspeitos são NPCs movidos por GPT. Continuação direta da CP4 (NLP, Chatbots e Agentes Virtuais, FIAP). Stack: Phaser.js no navegador, Python + FastAPI no servidor, API da OpenAI.

Como executar

1. Python 3.10 ou mais novo.
2. `python -m venv .venv` e ativar (`source .venv/bin/activate`, no Windows `.venv\Scripts\activate`).
3. `pip install -r requirements.txt`
4. Copie `.env.example` para `.env` e preencha `OPENAI_API_KEY` (não suba o `.env` para o GitHub).
5. `python run.py` e abra <http://127.0.0.1:8000>

Sem chave o jogo roda em modo offline (falas pré-escritas) e avisa na tela.

Como jogar

Cena do Crime: colete evidências (não gasta ação). Suspeitos: escolha com quem falar. Interrogatório: escreva a pergunta, escolha o tom (Perguntar, Pressionar, Acolher) e, se quiser, apresente evidências. São 4 ações por dia, 7 dias. Quando tiver provas, abra Estado do Caso e Montar acusação final.

Estrutura

- `backend/game.py` regras do jogo (sem rede), `backend/llm.py` integração com o GPT e filtro de saída, `backend/main.py` API.
- `frontend/ Phaser (js/main.js)`, arte procedural de reserva (`js/art.js`) e imagens opcionais em `assets/`.
- `tests/ testes das regras (python -m pytest -q)` e partida completa no navegador (`python tests/e2e_play.py`).
- `docs/prints/ capturas de tela da partida automatizada`.

Imagens geradas por IA (opcional)

Salve em `frontend/assets/`: `bg_menu.png`, `bg_room.png`, `portrait_beatriz.png`, `portrait_rafael.png`, `portrait_aurora.png`. O jogo usa os arquivos que existirem e desenha arte procedural no lugar dos que faltarem.

Publicar o jogo na web (ponto extra: build público)

Versão publicada: <https://jogo-fiap.onrender.com> (a primeira abertura pode levar cerca de 1 minuto).

O projeto já tem `Dockerfile` e `render.yaml`. No Render: New > Blueprint, aponte para o repositório do GitHub e preencha `OPENAI_API_KEY` no painel (nunca no código). A URL pública abre direto no menu do jogo. Variáveis opcionais: `MAX_GPT_CALLS_PER_SESSION` (padrão 40, depois disso o jogo usa falas pré-escritas), `MAX_SESSIONS` (300), `SESSION_TTL_SECONDS` (7200), `MAX_GPT_CALLS_PER_IP_DAY` (120), `MAX_GPT_CALLS_GLOBAL_DAY` (1500) e `MAX_NEW_GAMES_PER_IP_HOUR` (20), e `TRUST_PROXY=1` (já definido no `Dockerfile`, para ler o IP real atrás do proxy do Render). Os tetos por IP e global impedem que abrir várias partidas burle o limite por sessão. Perguntas têm no máximo 300 caracteres. No plano gratuito o servidor "dorme" e a primeira abertura pode levar cerca de 1 minuto.

Extras implementados

Testes automatizados (`python -m pytest -q`, 38 testes) e partida completa no navegador (`tests/e2e_play.py`, precisa de `pip install playwright && playwright install chromium`), fallback quando a API falha, filtro de saída, efeitos sonoros (botão de som no canto) e limites de custo. Documentação da entrega em `docs/ENTREGA_CP5.md`; prompts de imagem em `docs/prompts_imagens_nano_banana.md`.